



**Dr. D. Y. Patil Educational
Federation's
Dr. D. Y. Patil College of
Engineering and Innovation
Varale, Talegaon, Pune, 410507**
(Affiliated to Savitribai Phule Pune University)



AN INTERNSHIP REPORT ON Social Media Mutual Friend Analyzer

BY

**Vishal Shivraj Kumbhar– 13274, DIV- B
EXAM SEAT NO- T401230148**

Under the Guidance of

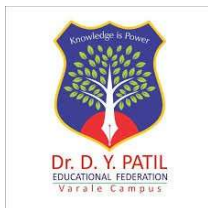
Dr. Alpana Adsul

In partial fulfillment of

T.E (COMPUTER ENGINEERING)

SAVITRIBAI PHULE PUNE UNIVERSITY

2025-2026



**Dr. D. Y. Patil Educational
Federation's
Dr. D. Y. Patil College of
Engineering and Innovation
Varale, Talegaon, Pune, 410507**

(Affiliated to Savitribai Phule Pune University)



CERTIFICATE

This is to certify that Mr. **Vishal Shivraj Kumbhar (Div. B, Roll No. 13274, Exam Seat No. T401230148)** is a bonafide student of the Department of Computer Engineering of Dr. DYPCOEI has submitted the necessary internship report titled **Social Media Mutual Friend Analyzer**.

The internship work has been carried out by him in a satisfactory manner under the supervision of the concerned faculty and is approved as a part of the partial fulfilment of the requirements of Savitribai Phule Pune University for the award of the degree of Bachelor of Engineering (Computer Engineering) for the academic year 2025-2026.

Dr. Alpana Adsul
(Internal Supervisor)

Mr. Mahendra Datta
(External Supervisor)

Prof. Anita Shinkar
(Internship Coordinator)

Dr. Alpana Adsul
HOD (COMP)

Dr. Suresh Mali
(Principal)

Date:

Place: Pune

INTERNSHIP OFFER LETTER

Academy of Skill Development

Module – 103, SDF Building,
Sector -V, Salt Lake,
Kolkata – 700091
www.asd.org.in



January 07, 2026

Ref: ASD/JAV/DR/88238

Subject: INTERNSHIP CONFIRMATION LETTER

Dear **VISHAL KUMBHAR**

Your application for the **Industrial Training and Internship** is accepted. The details are below:

College/University: DR. D. Y. PATIL COLLEGE OF ENGINEERING AND INNOVATION, VARALE, TALEGAON

Technology Domain: JAVA AND ITS APPLICATIONS (PROJECT BASED)

Internship Start Month: FEBRUARY 2026

Duration: 8 Weeks

This is a project based program. You will have to develop a project, prepare project report and project presentation.

★ **INTERNSHIP MILESTONES:** TRAINING ON THE TOPIC/TECHNOLOGY → PROJECT ALLOCATION → PROJECT IMPLEMENTATION → PROJECT REPORT PREPARATION /PPT PREPARATION → ASSESSMENT → CERTIFICATE DISBURSAL

Wish you a progressive learning journey with us.

Thanking you,

Best Wishes,

Mahendra Datta
Head – Learning and Development
Academy of Skill Development



INTERNSHIP OFFER LETTER

INTERNSHIP COMPLETION CERTIFICATE



ACADEMY OF SKILL DEVELOPMENT
A NON-PROFIT TRUST REGISTERED WITH GOVERNMENT OF WEST BENGAL U/S 60 AND RULE 69 (REGISTRATION NUMBER-190307248)
We dream of self-sufficient INDIA - हम आत्मनिर्भर भारत का सपना देखते हैं

Certificate of Completion



February 16, 2026 to March 30, 2026 (6 Weeks)

THIS CERTIFICATE IS HEREBY AWARDED TO
VISHAL SHIVRAJ KUMBHAR
OF
DR. D.Y. PATIL COLLEGE OF ENGINEERING AND INNOVATION, VARALE, TALEGAON
FOR THE SUCCESSFUL COMPLETION OF
INDUSTRIAL TRAINING AND INTERNSHIP
ON
JAVA AND ITS APPLICATIONS
AND IMPLEMENTING THE PROJECT TITLED
SOCIAL MEDIA MUTUAL FRIEND ANALYZER
WITH GRADE "A++".

Mahendra Satta.
Technical Head

Datta.
Operations Head

Certificate ID: ASD/JAV/DR./SOC/91452
Issue Date: March 30, 2026

Grading System
A++: 90% and above, A+: 75% to 89%, A: 65% to 74%, B: 55% to 64%

Ministry of MSME, Govt. of India
National Educational Alliance for Technology
ISO 9001:2015 CERTIFIED

Certificate Verification Link
<https://certificates.asd.org.in>

COMPANY/ORGANIZATION DETAILS

Background of the Organization



The Academy of Skill Development (ASD) is a non-profit, government-registered organization focused on skill development and employability enhancement. It operates as a unit of Ardent Computech Pvt. Ltd., an MSME and ISO 9001:2015 certified company. The organization is registered under the NGO/NPO category and is recognized with a Darpan ID. ASD aims to bridge the gap between academic learning and industry requirements by providing practical training programs. It offers industrial training, internships, and project-based learning opportunities for students and faculty across multiple disciplines. The organization collaborates with industry bodies and has a MoU with the AICTE NEAT Cell to deliver training programs. ASD is also an authorized Certiport test center for global certifications. Its courses are designed with strong industry involvement and updated according to current technological trends. The institution focuses on hands-on learning through live projects and real-world applications. Its mission is to develop industry-ready skills among candidates to meet dynamic job market demands. ASD also conducts workshops and webinars in educational institutions and corporate sectors. Overall, it works towards empowering youth and contributing to national initiatives like Digital India and skill development programs.

Scope and Objectives of the Internship

- ☐ **Implementation** – Focuses on applying Java concepts practically by developing real-world applications.
- ☐ **Development** – Involves designing, coding, and building software projects from scratch.
- ☐ **Testing** – Covers debugging and validating the application to ensure it works correctly.
- ☐ **Projects** – Includes working on industry-oriented, project-based tasks to gain hands-on experience.
- ☐ **Documentation** – Involves preparing reports and maintaining proper records of the project work.
- ☐ **Professionalism** – Helps in developing soft skills like teamwork, communication, and time management.

Objectives of the Internship

- **Learning** – To gain in-depth knowledge of Java and its real-world applications. It helps in understanding core and advanced concepts of programming. It builds a strong technical foundation for future development work.
- **Application** – To apply theoretical concepts in practical, project-based scenarios. It bridges the gap between classroom learning and industry requirements. It improves the ability to use concepts in real-time situations.
- **Analysis** – To understand problems and develop logical solutions effectively. It enhances critical thinking and problem-solving abilities. It trains the mind to approach complex tasks in a structured way.
 - **Execution** – To successfully implement and complete a software project. It focuses on converting ideas into working applications. It ensures proper planning, coding, and completion of tasks.
- **Improvement** – To enhance technical, coding, and problem-solving skills. It encourages continuous learning and skill development. It helps in identifying strengths and areas for improvement.
- **Readiness** – To prepare for industry requirements and professional work environments. It develops confidence to work in real job roles. It builds professional qualities like teamwork and communication skills.

Supervisor Details

- **Name:** Mahendra Datta
- **Designation:** Head – Learning and Development
- **Organization:** Academy of Skill Development (ASD)
- **Description:**

Mr. Mahendra Datta served as the supervisor for the internship program. He guided the training process, monitored project progress, and provided technical and professional support throughout the internship period.

ACKNOWLEDGEMENT

It is a great pleasure for me to acknowledge the assistance and contribution of a number of individuals who helped me in the Internship Project on “Energy Prediction and Management in Power BI”.

First and foremost, I wish to record my gratitude and thanks to **Dr. Alpana Adsul** (Internship Guide), for providing invaluable guidance throughout this internship. Their expertise and insights have significantly contributed to my learning experience. I am also thankful to **Academy of Skill Development(ASD)** for providing me with this opportunity to gain practical knowledge in a professional environment. Special thanks to **Mr. Mahendra Datta** (internship external supervisor) for their constant support and mentorship.

I express my thanks to **Dr. Suresh Mali** (Principal), **Dr. Alpana Adsul** (Head of Computer Engineering Department) for facilitating this internship program, and **Dr. Alpana Adsul**(Internship Coordinator) for their valuable guidance.

I am also thankful to the other teachers and non-teaching staff of the Computer Engineering Department and the Library for their cooperation and help.

Finally, I am grateful to my family and friends for their encouragement and moral support throughout this journey.

THANKING ALL OF YOU!

Vishal Shivraj Kumbhar

ABSTRACT

The **Social Media Mutual Friends Analyzer** is a Java-based application designed to model and analyze relationships in a social network using efficient data structures. The system stores each user's friends as a Set, ensuring uniqueness and preventing duplicate entries. It focuses on identifying mutual friends between users by applying Set Algebra operations, particularly intersection using the `retainAll()` method. The application demonstrates how mathematical concepts can be effectively used in programming to solve real-world problems. It provides a simple yet powerful way to manage and analyze social connections.

The project also highlights the use of **HashSet** for fast storage and **TreeSet** for automatically maintaining sorted results in alphabetical order. This eliminates the need for additional sorting logic, improving efficiency and code simplicity. The system is modular and easy to extend, making it suitable for learning concepts like data structures, algorithm design, and Java collections. Overall, the project serves as a practical implementation of Set-based operations in building a basic social network analyzer.

INDEX

Contents

Company/Organization Details	4
Acknowledgement	7
Abstract.....	8
1 Introduction.....	13
1.1 Problem Statement / Objectives.....	13
1.2 Motivation/Scope.....	14
2 Methodology.....	15
2.1 Proposed Solution	16
2.2 System Architecture.....	17
2.2.1 Dashboard Requirement Specification.....	18
2.2.2 Data Flow Architecture.....	18
2.3 Software Requirement Specification.....	19
Dashboard Requirement Specification.....	20
3 Results	22
3.1 Analysis.....	22
3.2 Inferences/Screenshots/Graph	22
4 Conclusion	25
5 List of References.....	26
Abbreviations	27
6 Appendix: Energy Prediction and Management	28
Appendix A: Software Setup	28
Appendix B: Data Preparation	28
Appendix C: Mutual Friend Analysis using Set Operations.....	29
Appendix D: Dashboard Features	29

Appendix E: Troubleshooting.....	29
----------------------------------	----

List of Figures

2.1	System Architecture Diagram	19
3.1	Dashboard	23
3.2	Network Graph	24

List of Tables

2.1 Requirement Specification.....	21
------------------------------------	----

Chapter 1

Introduction

1.1 Problem Statement / Objectives

This project aims to develop an easy-to-use system that analyzes social network relationships by identifying mutual friends between users using efficient data structures. As the number of users and connections increases, it becomes difficult to manually track and analyze common friendships. The system uses Set-based structures to ensure uniqueness of data and applies Set Algebra operations to quickly determine mutual connections. It helps in understanding relationships and managing social connections effectively.

The primary objectives of this project were:

- To design and develop a system that stores user and friend relationships using Set (HashSet/TreeSet) to avoid duplication.
- To implement Set Algebra operations, especially intersection using retainAll(), to efficiently find mutual friends.
- To provide quick and accurate analysis of user relationships using optimized data structures.
- To display mutual friends in sorted (alphabetical) order using TreeSet without manual sorting.
- To gain hands-on experience in Java collections, data structures, and algorithm implementation.
- To bridge theoretical knowledge with practical application by solving a real-world social network problem.

1.2 Motivation/Scope

In today's interconnected digital world, social networks contain a large number of users and complex relationships. Analyzing these relationships manually to identify mutual friends becomes difficult, time-consuming, and inefficient as the network grows. There is a need for a system that can quickly and accurately determine common connections without manual effort.

The motivation behind this project is to simplify the process of analyzing social relationships by using efficient data structures and algorithms. By applying Set-based operations, the system reduces computational complexity, eliminates duplicate data, and provides accurate results in less time. This approach helps in better understanding of social connections while improving efficiency and performance.

The scope of this project encompasses:

- **User Relationship Management:** The system allows users to create profiles and manage their friend connections efficiently.
- **Set-Based Data Handling:** Implementation of **Set (HashSet/TreeSet)** ensures uniqueness of friend data and avoids duplication.
- **Mutual Friend Analysis:** Use of **Set Algebra operations** such as intersection (`retainAll()`) to identify common friends between users.
- **Sorted Output Generation:** Utilization of **TreeSet** to automatically display mutual friends in alphabetical order without manual sorting.
- **Scalable Design:** The system can be extended to include advanced features such as friend suggestions, database integration, and graphical visualization.
- **Learning and Implementation:** Provides practical exposure to **Java collections, data structures, and algorithm design** in a real-world social network scenario.

Chapter 2

Methodology

2.1 Proposed Solution

To address the need for efficient analysis of social network relationships and identification of mutual connections, we propose the development of a **Social Media Mutual Friends Analyzer**, an intelligent system built using Java and its Collections Framework. This solution is designed to manage user relationships, eliminate duplicate data, and efficiently compute mutual friends using Set-based operations. The system can be applied in scenarios such as social networking platforms, recommendation systems, and connection analysis tools.

The system will function by storing user data and their corresponding friend lists using a `Map<String, Set<String>>` data structure. Each user's friends will be maintained in a **Set**, ensuring uniqueness and preventing duplicate entries. The data may be collected through user input (CLI or web interface) or loaded from stored files for persistence. The system will preprocess this data by converting all entries into a consistent format (e.g., lowercase usernames) to maintain uniformity and avoid redundancy.

A key component of the system is the implementation of **Set Algebra operations** to analyze relationships. To find mutual friends between two users, the system will use the `retainAll()` method, which performs the intersection of two sets efficiently. For improved output presentation, a **TreeSet** will be used to automatically sort the mutual friends in alphabetical order without requiring any additional sorting logic. This ensures both efficiency and readability of results.

Users will be able to interact with the system through a **command-line interface (CLI)** and optionally a **web-based interface** powered by a simple HTTP server. The interface will allow users to perform operations such as adding users, adding friends, viewing all connections, and finding mutual friends. The results will be displayed clearly, showing common connections and relationship insights in an understandable format.

Additionally, the system can be extended to provide **advanced analytical features**, such as friend suggestions using Set difference operations, identification of highly connected users, and detection of indirect relationships. Basic validation and error handling mechanisms will be implemented to manage cases such as non-existent users or invalid inputs, ensuring system reliability and robustness.

The methodology encompasses the following key steps:

- **Data Collection and Preprocessing: Data Collection and Preprocessing:**

The system collects user and friendship data through user input (CLI or web interface) or from stored files. Each user is associated with a list of friends, which is stored using a `Map<String, Set<String>>`. During preprocessing, usernames are

standardized (e.g., converted to lowercase), duplicates are removed automatically using Set properties, and invalid or inconsistent entries are handled to ensure clean and reliable data for processing.

- **Exploratory Data Analysis (EDA):** Basic analysis is performed on the social network data to understand relationships between users. This includes identifying users with maximum connections, analyzing common friendship patterns, and observing the distribution of connections. Logical inspection and simple visual representations (like printed structures or graphs) help in understanding how users are interconnected and in verifying the correctness of stored data.
- **Model Selection:** The core logic of the system is implemented using **Set Algebra operations**. To find mutual friends between two users, the intersection operation is applied using the `retainAll()` method. This ensures efficient computation of common elements. Additionally, operations like union (`addAll()`) and difference (`removeAll()`) can be used for advanced features such as friend suggestions and network analysis.
- **Dashboard Deployment:** The application provides a **Command Line Interface (CLI)** and can be extended to a **web-based interface using a simple HTTP server**. Users can interact with the system to add users, create friendships, view all connections, and find mutual friends. The interface is designed to be simple, interactive, and user-friendly.
- **Evaluation:** The system's performance is evaluated based on correctness, efficiency, and response time. The use of Set ensures no duplicate data and fast operations ($O(1)$ for `HashSet`). The accuracy of mutual friend results is verified through test cases, and the use of `TreeSet` ensures sorted and reliable output. The system is efficient for small to medium-sized datasets and demonstrates strong performance in handling social relationship queries.

2.2 System Architecture

The system architecture of the proposed project solution follows a streamlined process that converts user input data into meaningful relationship analysis. It begins with the collection of user and friendship data through a command-line interface (CLI) or web-based interface, followed by data loading and preprocessing steps such as standardizing usernames, removing inconsistencies, and ensuring uniqueness using Set data structures. Next, basic analysis of the stored data is performed to understand user connections and relationships. Based on this, Set Algebra operations are applied, particularly intersection using `retainAll()`, to identify mutual friends between users efficiently. The processed results are then organized and displayed using a `TreeSet`, which ensures that the output is always sorted in alphabetical order. Finally, the system presents the results through the interface, allowing users to view connections, analyze

relationships, and make better understanding of social networks in a simple and efficient manner.

2.2.1 Dashboard Requirement Specification

The dashboard for the proposed system must provide a clear and intuitive view of user relationships and social connections. It should display key information such as the list of users, their friends, and mutual connections between selected users. The interface should allow users to interactively perform operations like adding users, adding friends, and selecting two users to analyze their mutual friends.

The dashboard should prominently display the results of mutual friend analysis in a structured and easy-to-understand format, ensuring the output is sorted alphabetically using TreeSet. It should also provide options to view all user connections, highlight common relationships, and optionally suggest new friends based on existing connections. Simple visual elements such as lists or tables should be used to represent relationships clearly. Additionally, the system should support real-time updates and provide user feedback messages for successful operations or error handling, ensuring a smooth and user-friendly experience.

- **Visualization of Key Variables:** Display important social network information such as total users, number of friendships, and mutual friends using clear lists or simple visual representations.
- **Interactive Filtering:** Enable users to select and analyze specific users to view their friend lists and find mutual connections dynamically.
- **Model Output Integration:** Present the results of mutual friend analysis clearly, highlighting common friends between selected users in a structured and sorted format.
- **Trend and Correlation Analysis:** Show patterns in user connections, such as highly connected users or common friendship groups, using simple logical graph .

2.2.2 Data Flow Architecture

The data flow architecture of the proposed system begins with the collection of raw energy consumption data from sources such as smart meters, organizational logs, or simulated datasets. This data is then cleaned, transformed, and structured through preprocessing techniques to ensure it is suitable for analysis. The processed data is stored in secure storage systems for efficient access and management. Machine learning models are then applied to analyze the data, identify consumption patterns, and predict future energy usage and peak demand. The resulting insights are visualized through an interactive dashboard built using Power BI, enabling organizations to monitor performance and make informed decisions. The system is designed to continuously improve by incorporating new data and updating models to enhance prediction accuracy over time.

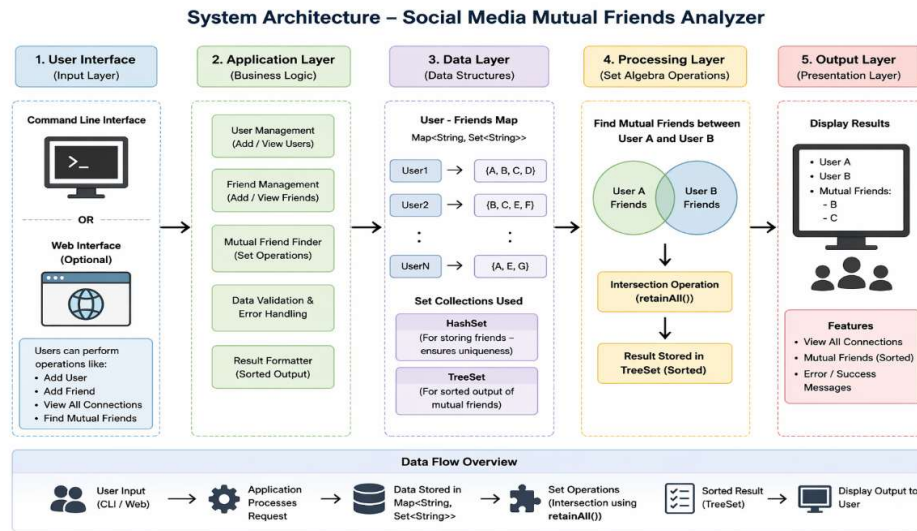


Figure 2.1: System Architecture Diagram

The architecture consists of the following key components:

Data Collection: Collect user and friendship data through user input using a Command Line Interface (CLI) or web interface. This component is responsible for gathering information about users and their connections in the social network.

Data Ingestion and Storage: Store the collected data in a structured format using Map<String, Set<String>>, where each user is mapped to a set of friends. The data can also be saved in files to ensure persistence and future retrieval.

Data Preprocessing: Prepare and clean the data by converting usernames into a consistent format (e.g., lowercase), removing invalid entries, and ensuring uniqueness using Set properties. This step improves data accuracy and consistency.

Set-Based Processing (Core Logic): Apply **Set Algebra operations** to analyze relationships. The system uses methods like retainAll() to find mutual friends (intersection), ensuring efficient and fast computation of common connections.

Sorted Output Handling: Use **TreeSet** to store and display results in alphabetical order automatically. This improves readability and eliminates the need for manual sorting.

2.3 Software Requirement Specification

A system designed to predict and analyze energy consumption for organizations such as

hospitals, data centers, and MNCs. It includes data acquisition from energy sources like smart meters and logs, preprocessing for data quality, and machine learning algorithms for predictive modeling. The system ensures efficient data handling, user-friendly dashboards for monitoring, and visualization tools for actionable insights. It also emphasizes scalability, reliability, and performance to support effective energy management and decision-making.

- **Dataset and Preprocessing:** The system will use energy consumption datasets (e.g., smart meter data, CSV logs). The data will be cleaned, processed, and transformed for analysis and machine learning.
- **Machine Learning Functionality:** The system will include models such as Linear Regression and Random Forest to predict future energy consumption and peak load patterns.
- **Dashboard Interface:** A user-friendly dashboard built using Power BI will allow users to view visualizations like energy trends, predicted vs actual usage, and key performance indicators.
- **Performance and Responsiveness:** The system will provide fast and efficient responses, ensuring smooth interaction and quick visualization updates even with large datasets.

Dashboard Requirement Specification

Table 2.1: Requirement Specification

Req. ID	Component	Description	Priority	Status
REQ-001	Data Input	Accept user and friend data via CLI or web interface	High	Completed
REQ-002	Data Cleaning	Standardize usernames (lowercase), remove duplicates using Set	High	Completed
REQ-003	Data Structure	Store data using <code>Map<String, Set<String>></code> for efficient access	High	Completed
REQ-004	Dashboard Layout	Create simple and intuitive UI for user interaction	High	Completed
REQ-005	User List Display	Show all users and their friend connections	High	Completed
REQ-006	Mutual Friends	Display mutual friends between two selected users	High	Completed
REQ-007	Sorted Output	Use <code>TreeSet</code> to display results in alphabetical order	High	Completed
REQ-008	Friend Addition	Allow adding new friends to existing users	High	Completed
REQ-009	User Addition	Allow creation of new users	High	Completed
REQ-010	Error Handling	Handle invalid inputs and non-existing users	High	Completed
REQ-011	Suggestion Feature	Suggest friends using Set difference logic	Medium	Optional
REQ-012	Deployment	Run system via CLI or local web server	High	Completed

Chapter 3

Results

3.1 Analysis

The Social Media Mutual Friends Analyzer effectively identifies relationships and patterns within user connections using structured data representation and logical operations. The system analyzes user friendships using Set-based data structures, allowing efficient identification of mutual friends between users. This approach helps in understanding how users are interconnected and highlights common relationships within the network.

The data preprocessing phase ensures consistency by standardizing usernames, removing duplicate entries, and handling invalid inputs. The use of Set automatically eliminates redundancy, improving the accuracy and reliability of the analysis. Logical inspection of user connections enables clear visualization of relationships, helping users easily interpret how friendships are distributed across the network.

The implementation of Set Algebra operations, particularly intersection using `retainAll()`, enhances the analytical capability of the system by efficiently computing mutual friends. Additionally, the use of `TreeSet` ensures that results are displayed in a sorted and readable format. Overall, the system provides a simple yet effective, data-driven approach to analyzing social connections, enabling users to understand relationships, identify common links, and manage networks efficiently.

3.2 Inferences/Screenshots/Graph

The system's functionality was evaluated through an interactive dashboard developed using Power BI, designed to provide insights into energy consumption patterns and predictive trends. Users can interact with the dashboard by applying filters such as date, time, department, or location to dynamically explore the data and observe variations in energy usage. Visual components such as line charts, bar graphs, pie charts, and heatmaps enhance data interpretation and help identify key patterns — for example, higher energy consumption during peak operational hours or increased usage correlated with temperature changes.

The dashboard also highlights comparisons between predicted and actual energy consumption, enabling users to evaluate model performance and detect anomalies. KPI cards provide quick insights into total consumption, peak load, and estimated costs. These visualizations collectively help organizations understand energy behavior, optimize usage, and make data-driven decisions for improved efficiency.

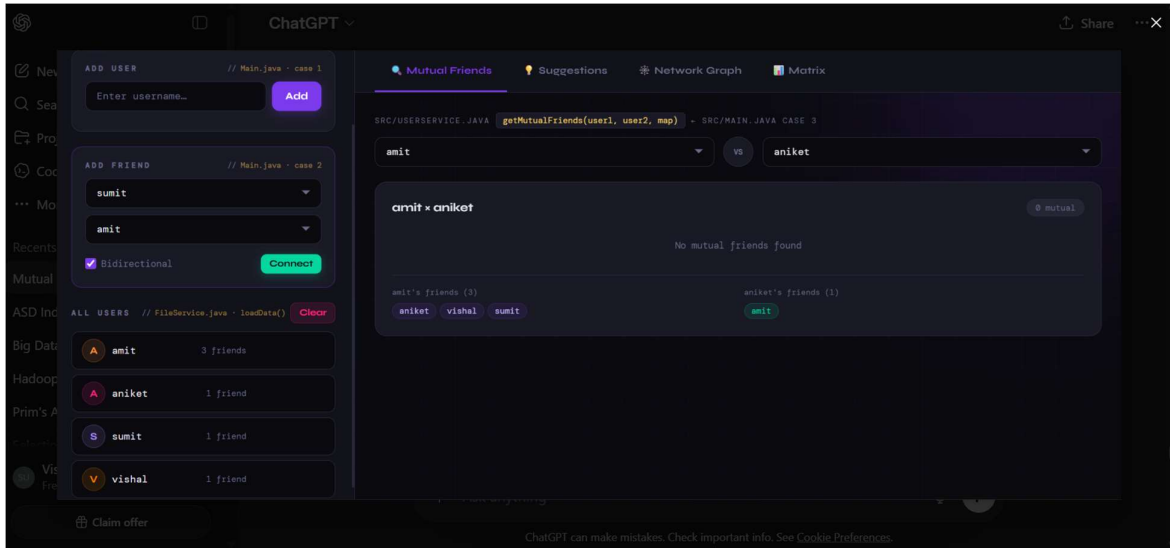


Figure 3.1: Dashboard

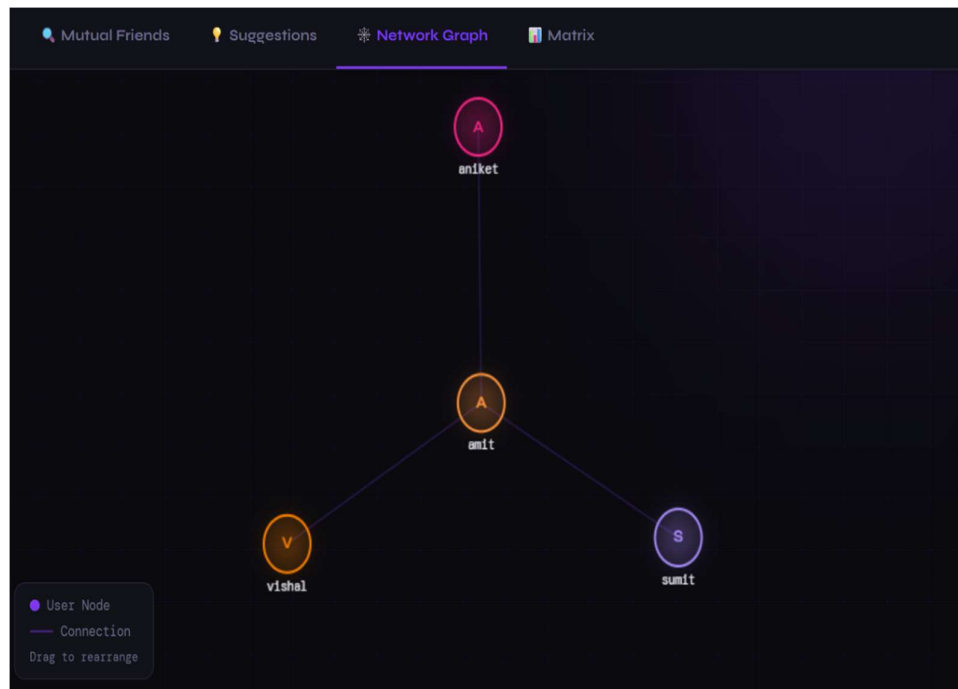


Figure 3.2: Actual vs Predicted Energy Demand

Data preprocessing and transformation were performed before generating the network graph, ensuring that user data was clean, consistent, and free from duplicate entries. Usernames were standardized and stored using Set-based structures, which automatically removed redundancy and maintained uniqueness. The graph layout was designed for clarity, where each user is represented as a node and each friendship is shown as a connecting edge. Proper spacing and visual alignment were used to make the network easy to understand and interpret.

During development, the network graph was used to visualize relationship patterns among users. A key observation from the graph is the identification of highly connected users, such as central nodes with multiple connections, and less connected or isolated users. The graph clearly highlights how users are linked and whether mutual connections exist between them. This visual representation helps in understanding the structure of the social network, identifying connection clusters, and analyzing relationship distribution. Overall, the graph supports logical conclusions about user connectivity and enhances the system's ability to present social relationship insights effectively.

Chapter 4

Conclusion

The Social Media Mutual Friends Analyzer successfully integrates efficient data structures and logical operations to provide meaningful insights into user relationships and network connectivity. By leveraging Set-based data handling and Set Algebra operations, the system enables users to analyze friendships, identify mutual connections, and understand relationship patterns effectively.

The system is intuitive and user-friendly, making it accessible to both technical and non-technical users. Through features such as structured user lists, mutual friend analysis, and network graph visualization, the application clearly represents relationships between users. It highlights key aspects such as highly connected users and the absence or presence of mutual connections, helping users better understand the structure of the social network. Edge cases, such as invalid inputs or duplicate entries, are handled efficiently using Set properties, ensuring the system remains reliable and accurate. Additionally, the system demonstrates efficient performance with fast operations and organized output.

In conclusion, the Social Media Mutual Friends Analyzer serves as a practical and efficient tool for analyzing social relationships and identifying common connections. It simplifies complex relationship analysis using optimized data structures and provides clear, sorted results. The system also offers strong potential for future enhancements such as database integration, advanced friend suggestion algorithms, and real-time web deployment.

This project has been a valuable hands-on learning experience, contributing to a deeper understanding of data structures, Set operations, and Java-based application development. It provided practical exposure to solving real-world problems using programming concepts, strengthening skills in algorithm design, system development, and logical problem-solving.

Chapter 5

List of References

- Oracle Corporation. (2023). *Java Platform, Standard Edition Documentation*. <https://docs.oracle.com/javase/>
- Bloch, J. (2018). *Effective Java (3rd Edition)*. Addison-Wesley Professional.
- Horstmann, C. S. (2019). *Core Java Volume I – Fundamentals*. Pearson Education.
- Oracle Corporation. (2023). *Java Collections Framework (Set, HashSet, TreeSet)*. <https://docs.oracle.com/javase/tutorial/collections/>
- Schildt, H. (2020). *Java: The Complete Reference (11th Edition)*. McGraw-Hill Education.
- Mozilla Developer Network (MDN). (2023). *HTTP and Web APIs*. <https://developer.mozilla.org>
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

ABBREVIATIONS

DS Data Structure

CLI Command Line Interface

GUI Graphical User Interface

UI User Interface

UX User Experience

API Application Programming Interface

HTTP HyperText Transfer Protocol

URL Uniform Resource Locator

JSON JavaScript Object Notation

DB Database

JCF Java Collections Framework

OOP Object-Oriented Programming

MAP Key-Value Data Structure

SET Collection of Unique Elements

HS HashSet

TS TreeSet

EDA Exploratory Data Analysis

SRS Software Requirement Specification

Chapter 6

Appendix: Energy Prediction and Management

Appendix A: Software Setup

Step 1: Install Required Tools

Ensure you have the following installed:

- Java Development Kit (JDK 8 or above)
- Any Java IDE (IntelliJ IDEA / Eclipse / VS Code)
- Web Browser (Chrome / Edge) for GUI (optional)

Step 2: Setup the Project

Create a new Java project in your IDE

- Add required classes such as Main.java, UserService.java, and FileService.java
- Place index.html file in the project directory (for web interface)
- Compile and run the Main.java file

Appendix B: Data Preparation

Data Handling

User and friend data is collected through CLI or web interface

- Data is stored using Map<String, Set<String>>
- Each user is associated with a unique set of friends

Data Cleaning

- Convert all usernames to lowercase for consistency
 - Remove duplicate friend entries automatically using Set
 - Validate inputs to avoid invalid or empty values

Data Organization

- Maintain structured mapping of users and friends
- Ensure proper relationship linking between users
- Save and load data using file handling for persistence

Appendix C: Mutual Friend Analysis using Set Operations

Mutual Friend Calculation

- Use Set intersection to find common friends
- Apply retainAll() method for efficient computation

Example:

```
Set<String> mutual = new TreeSet<>(userFriendsMap.get(user1));  
mutual.retainAll(userFriendsMap.get(user2));
```

Friend Management

- Add new users
- Add and remove friendships
- Maintain unique friend lists using Set

Appendix D: Dashboard Features

CLI Features

- Add User
- Add Friend
- Find Mutual Friends
- Display All Users

Web Interface Features (Optional)

- User-friendly dashboard accessible via browser (localhost:8080)
- Forms to add users and friends
- Display of mutual friends in sorted order
- Real-time interaction with backend

Visualization Features

- Display user relationships in structured format
- Network graph showing connections between users
- Highlight mutual friends clearly

Appendix E: Troubleshooting

Application Issues

- Issue: Program not running

Solution: Ensure JDK is installed and environment variables are set

- Issue: Port 8080 already in use

Solution: Change port number in code or close conflicting application

- Issue: Data not saving

Solution: Check file path and permissions for FileService

Logic Issues

- Ensure users exist before adding friends
- Verify correct use of retainAll() for mutual friends
- Check for null or empty inputs

Performance Tips

- Use HashSet for faster operations
- Use TreeSet only for sorted output
- Avoid unnecessary loops by using built-in Set method

